PRAKTEK TEKNOLOGI IOT II

LAPORAN PRAKTEK

TEKNOLOGI IOT II

Disusun dalam rangka memenuhi salah satu syarat kelengkapan nilai Praktikum

Mata Kuliah Teknologi IOT II

DISUSUN OLEH :

NAMA : Davina Anisa

NPM 25782007

PRODI : TEKNOLOGI REKAYASA INTERNET

KELAS : 3A

DOSEN : Dr. ir. Septafiansyah Dwi Putra, S. T.,M.T.

: Agus Ambarwari, S.Pd.,M.Kom.

4.7 Tugas: Sistem Kendali Intensitas Cahaya (PWM Slider) via *WebSockets*

Sinyal Digital (*ON/OFF*) sangat berguna, namun bagaimana jika Anda ditugaskan merancang antarmuka penyesuai intensitas redup-terang lampu (*Dimmer*) dari *smartphone*?

Skenario Proyek (*Smart Lighting*): Anda akan mengeksplorasi penggunaan modulasi sinyal listrik analogWrite() (atau dikenal dengan fitur PWM) dan memadukannya dengan teknologi masukan elemen *Slider / Range* dari HTML5.

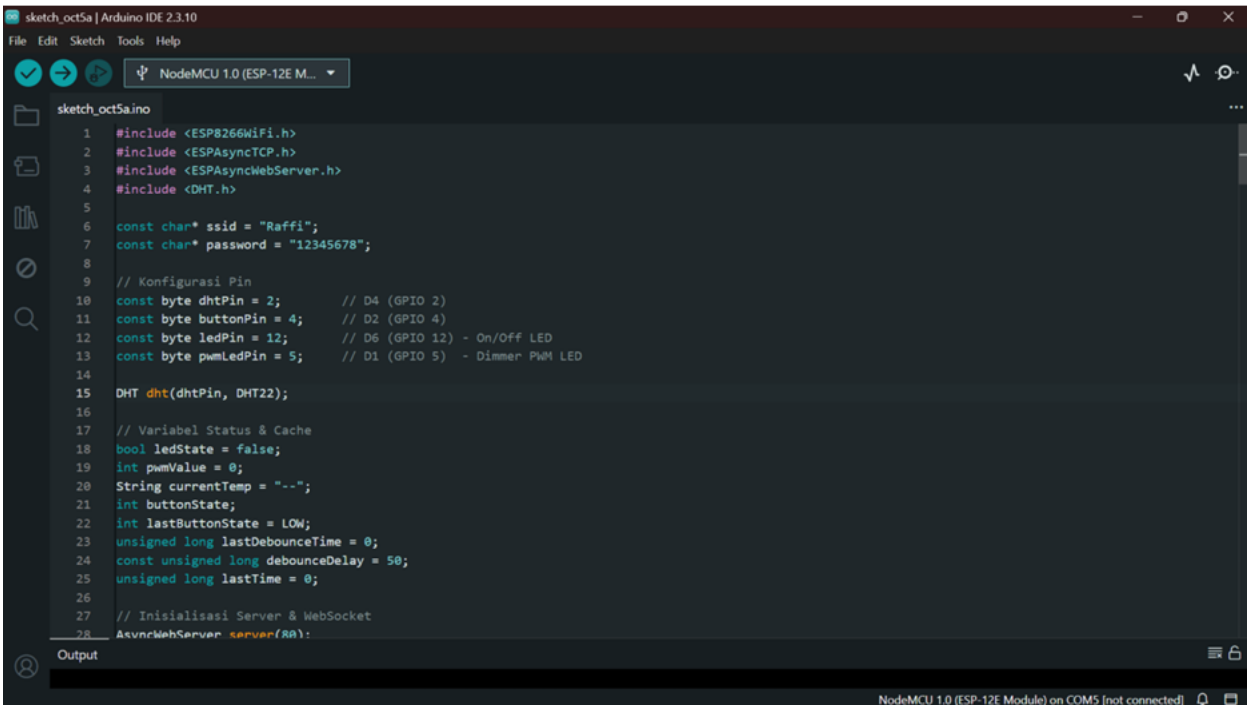
- **Tujuan:** Menguasai transmisi paket *String* WebSocket berkelanjutan (*continuous packet*) dari manipulasi antarmuka layar *slider* untuk merombak parameter persentase daya komponen aktuator.
- **Tingkat Kesulitan:** Lanjut
- **Instruksi Perangkaian:**
    i. Tambahkan sebuah komponen **LED** yang dilengkapi **Resistor 220 Ohm** (sebagai pengaman arus).
    ii. Hubungkan kaki Anoda (positif) sistem LED tersebut ke pin **D1 (GPIO 5)**.
- **Instruksi Pemrograman (Front-End & Back-End):**
    i. Modifikasi desain kerangka web HTML Anda. Sisipkan elemen input *Slider* HTML. *(Hint sintaksis: <input type="range" min="0" max="1023" id="pwmSlider" onchange="sendPWM(this.value)">)*
    ii. Buat sebuah fungsi JavaScript bernama sendPWM(value) yang akan mengirimkan paket *string* teks berformat khusus (misalnya "pwm,512") ke NodeMCU melalui *websocket*.
    iii. Pada ruang fungsi C++ handleWebSocketMessage() di sisi server (NodeMCU), pecah (*parsing*) atau periksa string tersebut menggunakan fitur deteksi *substring* atau bahasa pemrosesan C++ lainnya.
    iv. Jika Server NodeMCU sukses mendeteksi dan mengambil angka murninya, panggil fungsi aktuator analogWrite(pinLED, nilaiPWM) untuk secara elegan meredupkan atau menerangkan LED LED fisik secara seketika sesuai gesekan layar Anda!
- **Kriteria Keberhasilan:**
    - Di layar gawai Anda, hadir tuas gulir horizontal (*Slider*).
    - LED di dunia nyata dapat diredupkan dan diterangkan secara bertahap saat Anda menggeser tuas *slider*, bebas dari lag (*real-time response*).
    - Kendali LED dan pembacaan Sensor *DHT* bawaan Praktikum 1 dipastikan masih bekerja berdampingan dengan sempurna.

[PENTING!]

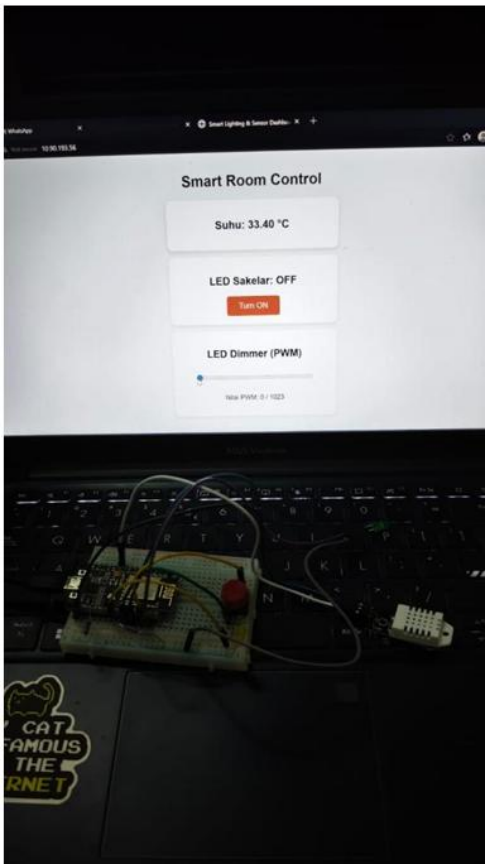
Instruksi Pengumpulan Tugas Praktikum Seluruh pengerjaan praktikum dan tugas (meliputi *source code* berformat .ino, dokumentasi foto/video sirkuit yang berhasil, tangkapan layar *Serial Monitor*, serta laporan praktikum tertulis .PDF) harus dikumpulkan dengan cara melakukan **commit dan push ke repositori GitHub pribadi Anda masing-masing**.

Tautkan/kumpulkan *URL repositori GitHub* Anda pada sistem manajemen pembelajaran (LMS) kampus sebagai bukti penyelesaian praktikum ini.

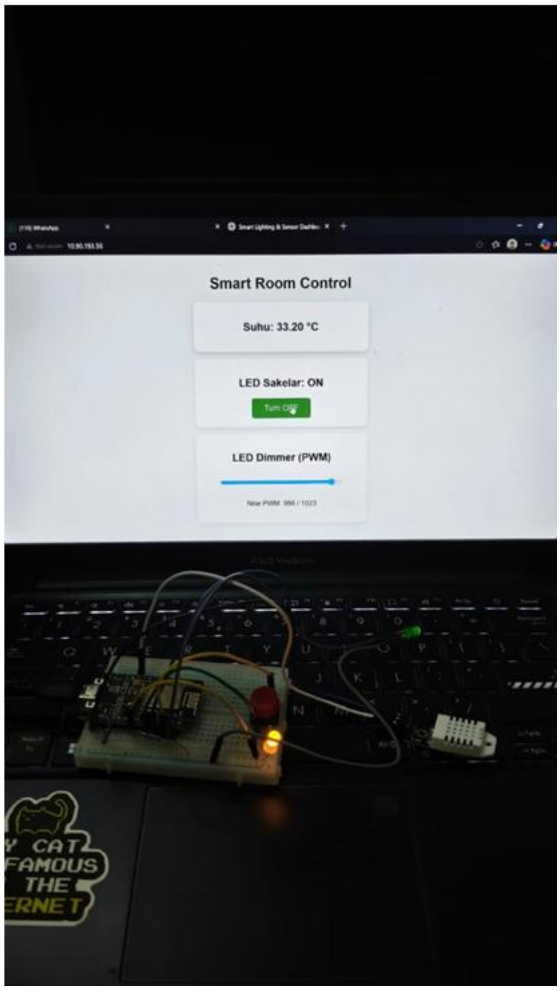
KODE DAN RANGKAIAN :



HASIL LED KETIKA OFF :



HASIL LED KETIKA ON :



NAMA KELOMPOK :

1. Muhammad Romzi – 25782015

2. Muhammad Daffa Al-Barru – 25782017

3. Muhammad Khoiru Nizar – 25782018

4. Raffi Maulana Abdillah – 25782021

5.7 Tugas: Sistem Telemetri & Otomasi Terdistribusi Berbasis painlessMesh

Kembangkan jaringan mesh pada Praktikum 4 menjadi sistem *monitoring* lingkungan dan kendali otomasi terdistribusi tanpa perantara router. Setiap kelompok membagi peran perangkat sebagai berikut:

- Tujuan: Menguasai transmisi paket JSON terdistribusi melintasi jaringan *Ad-Hoc Mesh Multi-Hop* dan mengevaluasi *rule engine* otomasi aktuator LED di node konsentrator.
- Tingkat Kesulitan: Lanjut (Kolaboratif Kelompok)
- Instruksi Perangkaian:

i. Node 1 (Stasiun Lingkungan): Hubungkan pin DATA sensor DHT11/DHT22 ke pin D4 (GPIO 2) NodeMCU 1 (VCC ke 3V3, GND ke GND).

ii. Node 2 (Stasiun Cahaya): Rangkai sensor LDR dan Resistor 10k Ohm sebagai pembagi tegangan ke pin A0 NodeMCU 2.

iii. Node 3 (Hub Kontrol Aktuator):

- Hubungkan kaki Anoda (kaki panjang) LED ke pin D6 (GPIO 12) NodeMCU 3.
- Hubungkan kaki Katoda (kaki pendek) LED ke salah satu kaki Resistor 220 Ohm, lalu hubungkan kaki resistor lainnya ke jalur GND *(rangkaian LED aktif-HIGH seperti pada Modul 3 dan Modul 4)*.

- Instruksi Pemrograman (Front-End & Back-End Mesh):

i. Node 1 (Sensor DHT):

- Modifikasi fungsi sendMessage() agar membaca suhu dan kelembapan riil menggunakan pustaka DHT.h setiap 3 detik.
- Kemas data ke dalam format string JSON ringkas dan siarkan via mesh:

{"tipe":"suhu_node","suhu":29.4,"kelembapan":65.0}

ii. Node 2 (Sensor LDR):

- Modifikasi fungsi sendMessage() agar membaca nilai ADC analog LDR setiap 3 detik.
- Kemas data ke dalam format string JSON ringkas dan siarkan via mesh:

{"tipe":"cahaya_node","adc":420}

iii. Node 3 (Hub Aktuator Cerdas):

- Nonaktifkan taskSendMessage pada Node 3 (node ini bertindak pasif sebagai penerima dan pengambil keputusan).
- Pada fungsi receivedCallback(uint32_t from, String &msg):
  - Gunakan pustaka ArduinoJson (JsonDocument / StaticJsonDocument<200>) untuk membedah string JSON masuk.
  - Evaluasi kondisi *Rule Engine*:
    - JIKA data suhu dari Node 1 bernilai di atas 31.0 Celcius ATAU nilai ADC cahaya dari Node 2 bernilai di bawah 300 (kondisi gelap), nyalakan LED (digitalWrite(ledPin, HIGH)).
    - Jika kedua parameter kembali ke kondisi normal, matikan LED (digitalWrite(ledPin, LOW)).

iv. Eksperimen Ketahanan Jaringan (*Multi-Hop* dan *Self-Healing*):

- Jauhkan Node 1 dan Node 3 hingga berada di luar jangkauan radio langsung masing-masing.
- Tempatkan Node 2 di posisi tengah antara Node 1 dan Node 3.

        - Amati pada Serial Monitor apakah paket telemetri dari Node 1 tetap dapat diterima oleh Node 3 melalui relai perantara (*multi-hop relay*) Node 2.
        - Matikan daya Node 2 (simulasi node putus/gagal), lalu amati perubahan status pada Serial Monitor Node 3 melalui *callback* changedConnectionCallback().

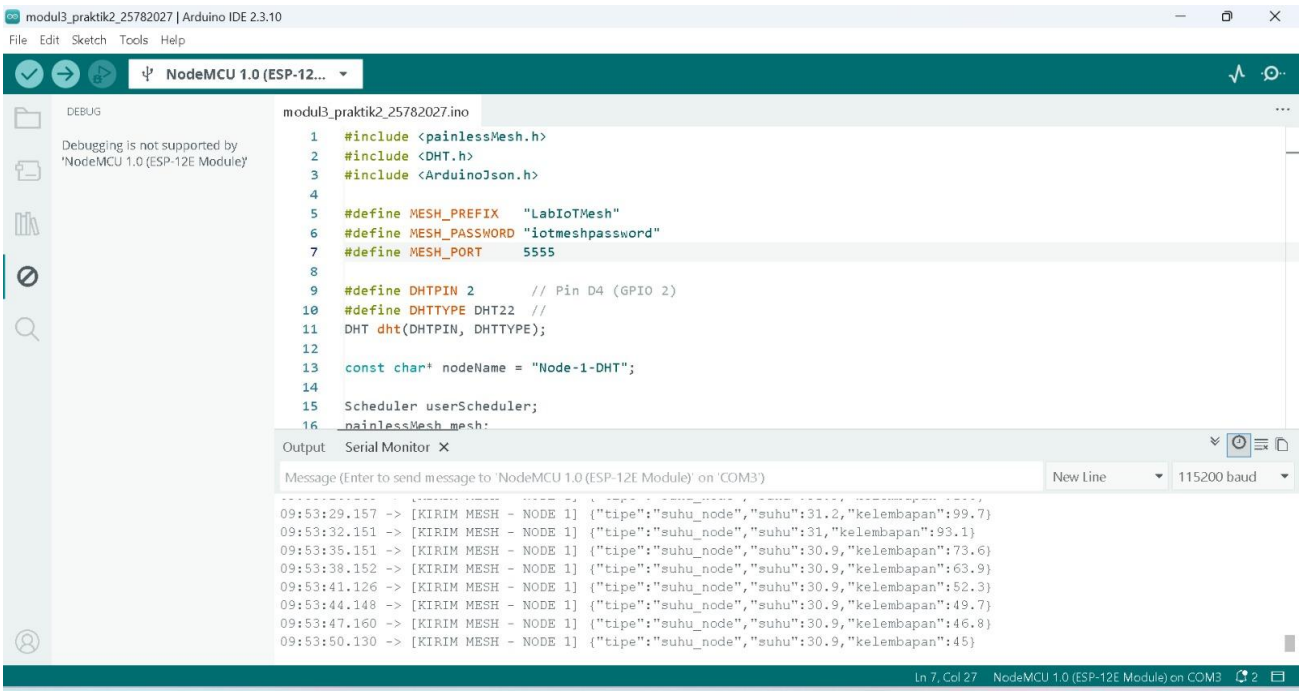
- Kriteria Keberhasilan:
    - Ketiga node berhasil membentuk jaringan rantai mesh otonom tanpa menggunakan router Wi-Fi eksternal.
    - Data sensor riil dari Node 1 dan Node 2 berhasil diterima dan di-*parsing* dengan benar oleh Node 3 menggunakan format JSON.
    - Logika *Rule Engine* pada Node 3 berhasil memicu aktuator LED fisik sesuai kondisi ambang batas (*threshold*).
    - Mekanisme *Multi-Hop Relay* dan *Self-Healing* teramati dengan jelas melalui log Serial Monitor.

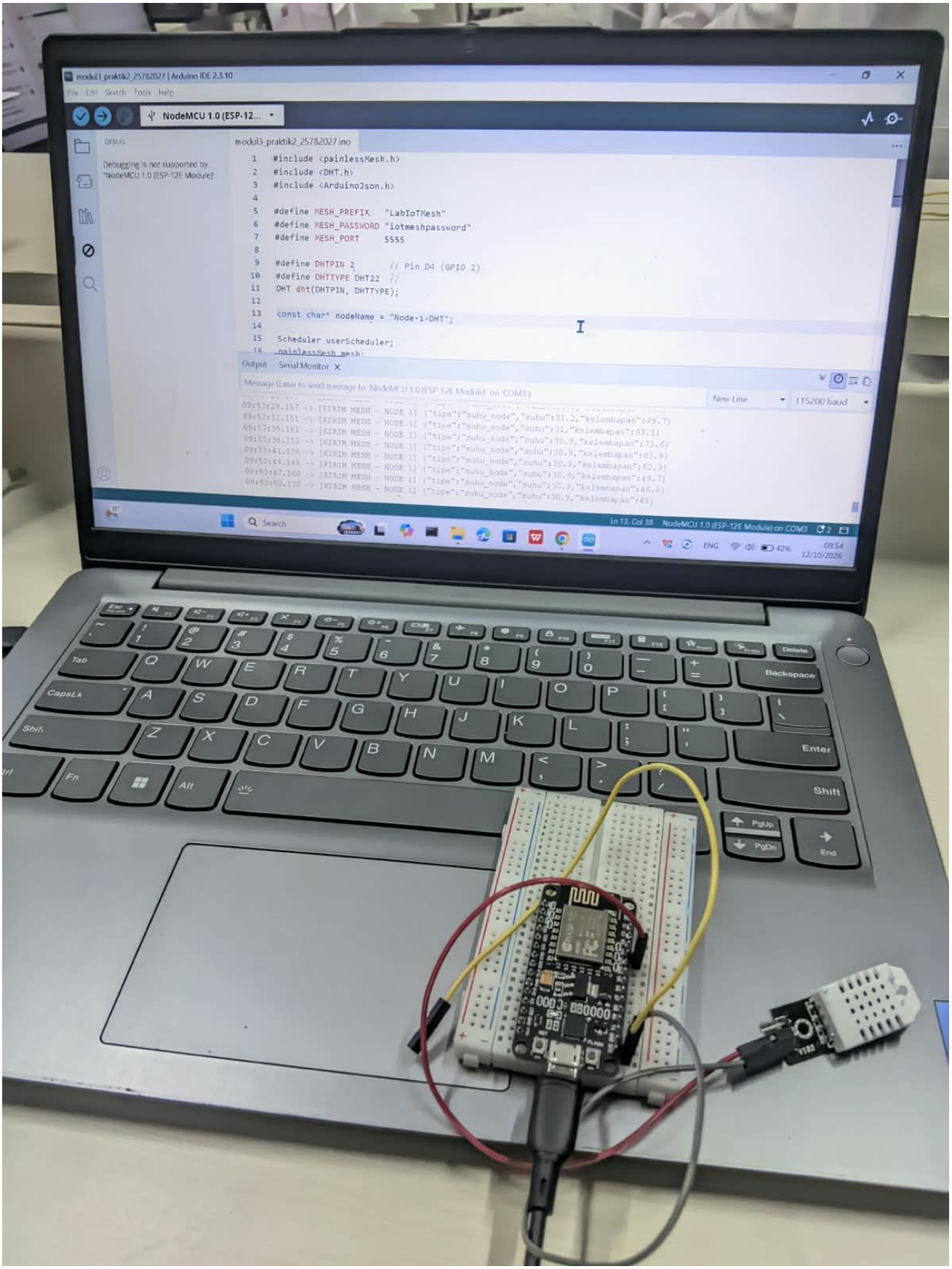
[PENTING!] Instruksi Pengumpulan Tugas Praktikum Seluruh pengerjaan praktikum dan tugas (meliputi *source code* berformat .ino, dokumentasi foto/video sirkuit yang berhasil, tangkapan layar *Serial Monitor*, serta laporan praktikum tertulis .PDF) harus dikumpulkan dengan cara melakukan commit dan push ke repositori GitHub pribadi Anda masing-masing.

Tautkan/kumpulkan *URL repositori GitHub* Anda pada sistem manajemen pembelajaran (LMS) kampus sebagai bukti penyelesaian praktikum ini.

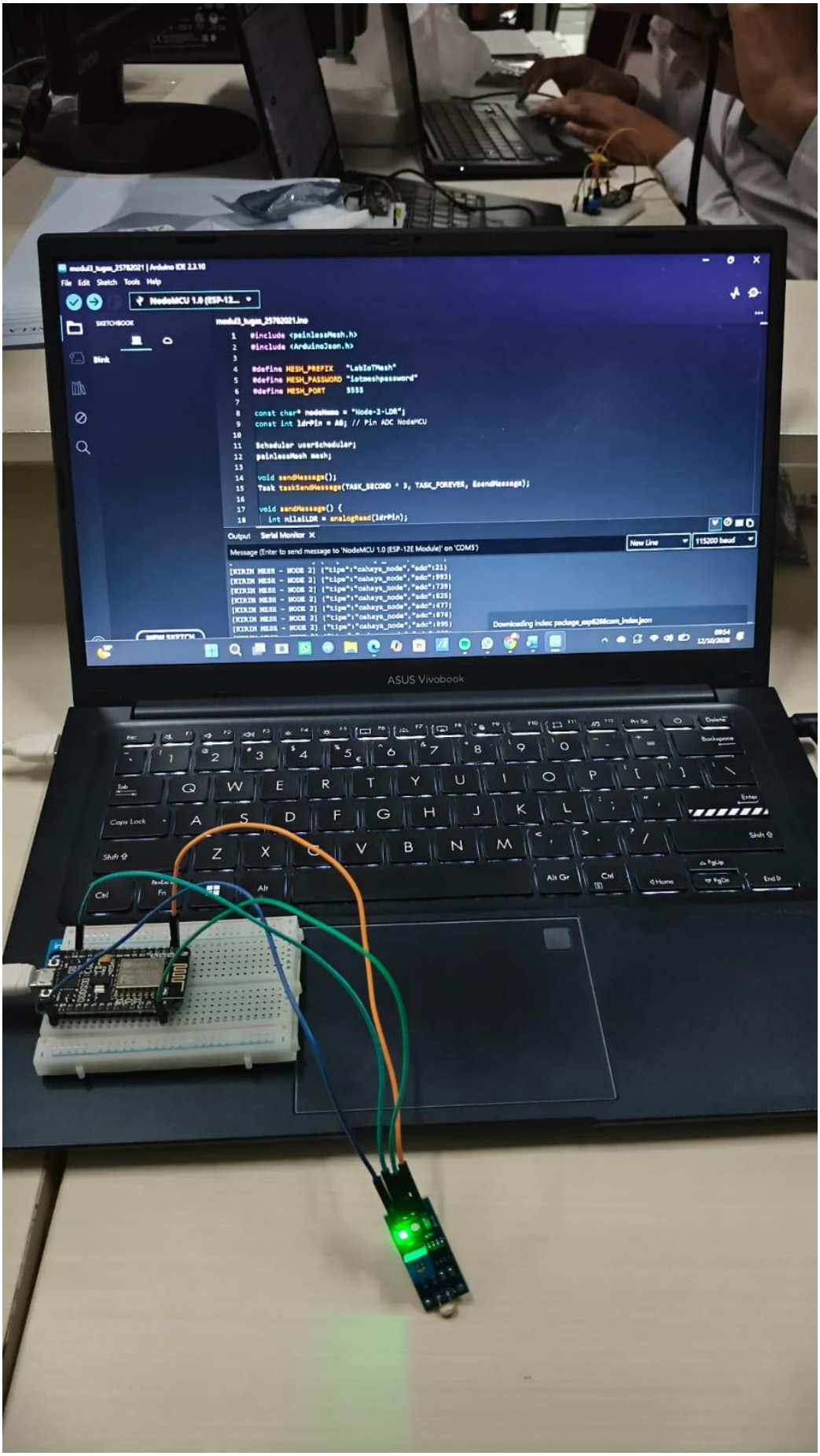
KODE & RANGKAIAN :

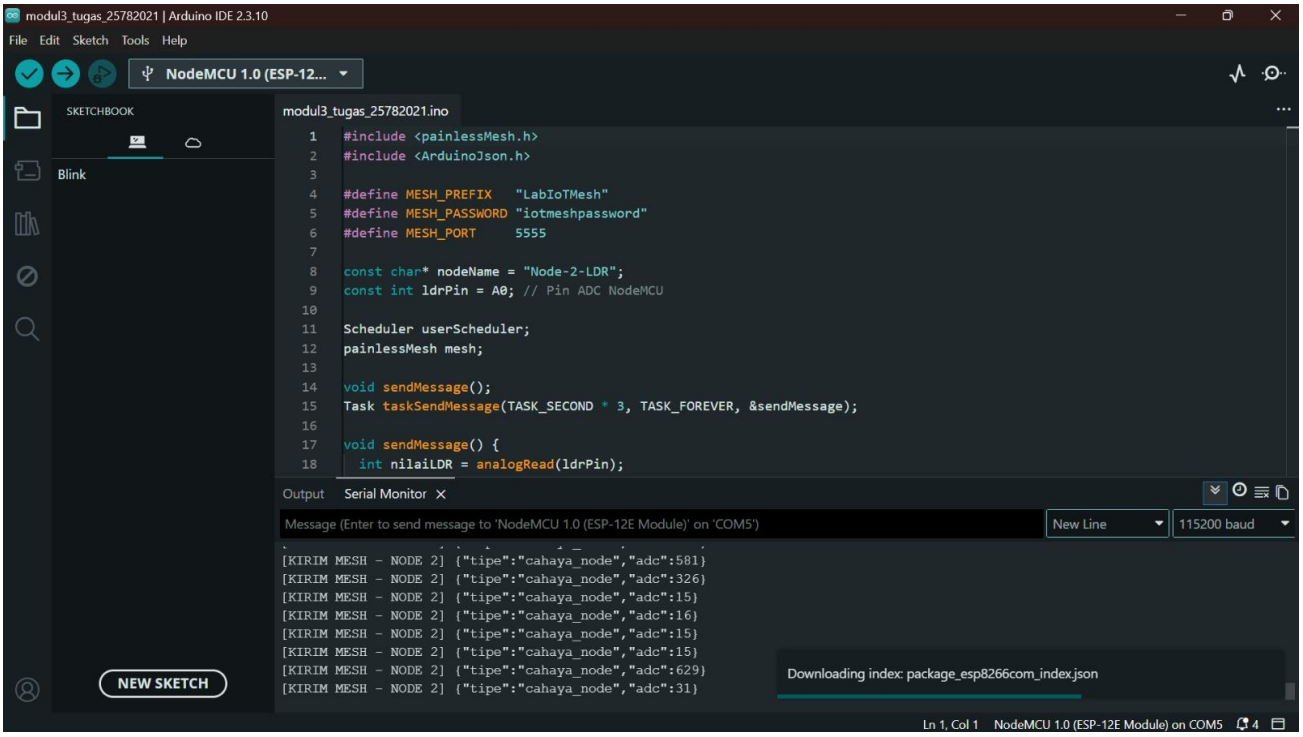
NODE 1 :



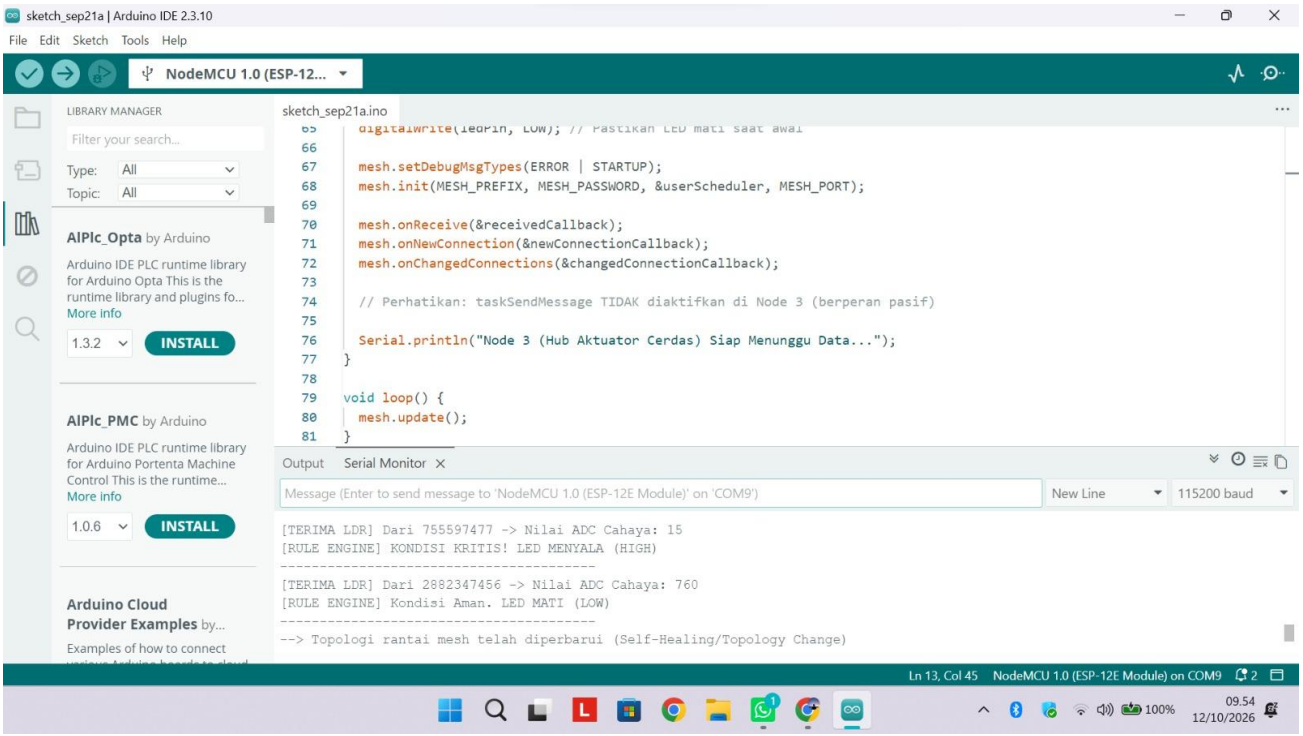

modul3_praktik2_25782027 | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
DEBUG
Debugging is not supported by "NodeMCU 1.0 (ESP-12E Module)"
modul3_praktik2_25782027.ino
#include <painlessMesh.h>
#include <DHT.h>
#include <ArduinoJson.h>
#define MESH_PREFIX "LabIoTMesh"
#define MESH_PASSWORD "iotmeshpassword"
#define MESH_PORT 5555
#define DHTPIN 2 // Pin D4 (GPIO 2)
#define DHTTYPE DHT22 //
DHT dht(DHTPIN, DHTTYPE);
const char* nodeName = "Node-1-DHT";
Scheduler userScheduler;
painlessMesh mesh;
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')
New Line
115200 baud
09:53:29.157 -> [KIRIM MESH - NODE 1] {"tipe":"suhu_node","suhu":31.2,"kelembapan":99.7}
09:53:32.151 -> [KIRIM MESH - NODE 1] {"tipe":"suhu_node","suhu":31,"kelembapan":93.1}
09:53:35.151 -> [KIRIM MESH - NODE 1] {"tipe":"suhu_node","suhu":30.9,"kelembapan":73.6}
09:53:38.152 -> [KIRIM MESH - NODE 1] {"tipe":"suhu_node","suhu":30.9,"kelembapan":63.9}
09:53:41.126 -> [KIRIM MESH - NODE 1] {"tipe":"suhu_node","suhu":30.9,"kelembapan":52.3}
09:53:44.148 -> [KIRIM MESH - NODE 1] {"tipe":"suhu_node","suhu":30.9,"kelembapan":49.7}
09:53:47.160 -> [KIRIM MESH - NODE 1] {"tipe":"suhu_node","suhu":30.9,"kelemapan":46.8}
09:53:50.130 -> [KIRIM MESH - NODE 1] {"tipe":"suhu_node","suhu":30.9,"kelembapan":45}
Ln 13, Col 38 NodeMCU 1.0 (ESP-12E Module) on COM3
Search
ENG
42%
09:54
12/10/2026

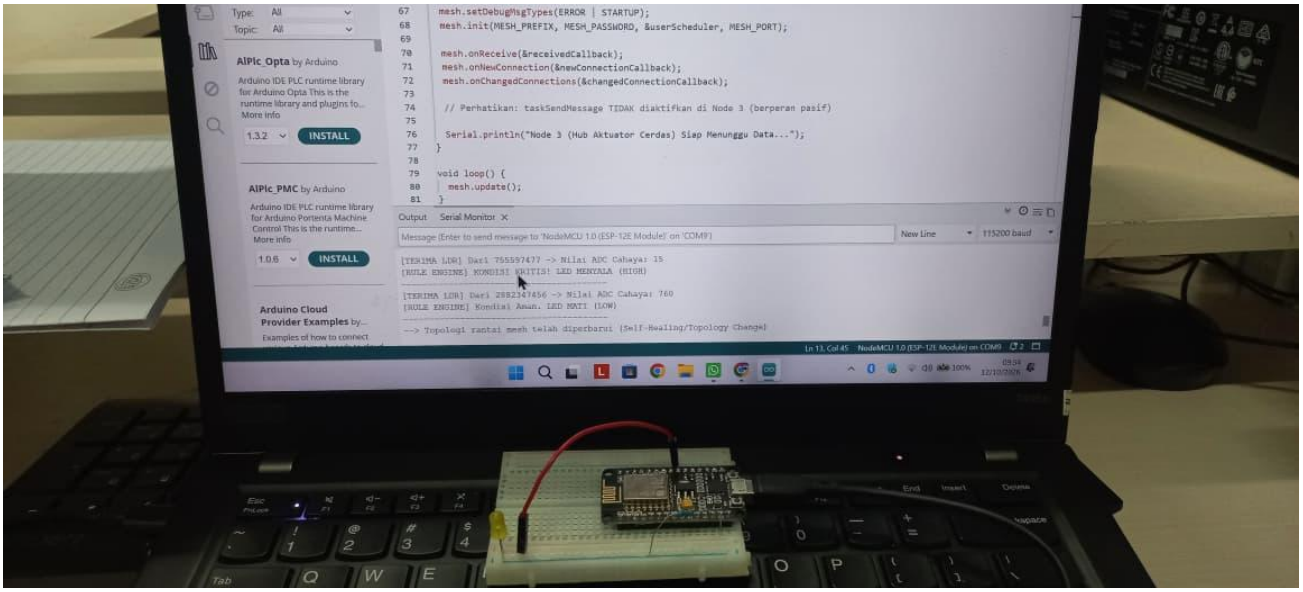
NODE 2 :


File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
#include <painlessMesh.h>
#include <ArduinoJson.h>
#define MESH_PREFIX "LabIoTMesh"
#define MESH_PASSWORD "iotmeshpassword"
#define MESH_PORT 5555
const char* nodeName = "Node-1-LDR";
const int ldrPin = A0; // Pin ADC NodeMCU
Scheduler userScheduler;
painlessMesh mesh;
void sendMessage();
Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
void sendMessage() {
int nilaiLDR = analogRead(ldrPin);
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')
New Line
115200 baud
[KIRIM MESH - NODE 2] {"tipe":"cahaya_node","adc":121}
[KIRIM MESH - NODE 2] {"tipe":"cahaya_node","adc":993}
[KIRIM MESH - NODE 2] {"tipe":"cahaya_node","adc":739}
[KIRIM MESH - NODE 2] {"tipe":"cahaya_node","adc":825}
[KIRIM MESH - NODE 2] {"tipe":"cahaya_node","adc":477}
[KIRIM MESH - NODE 2] {"tipe":"cahaya_node","adc":878}
[KIRIM MESH - NODE 2] {"tipe":"cahaya_node","adc":899}
Downloading index: package_esp8266com_index.json
ASUS Vivobook


modul3_tugas_25782021 | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
SKETCHBOOK
Blink
NEW SKETCH
modul3_tugas_25782021.ino
#include <painlessMesh.h>
#include <ArduinoJson.h>
#define MESH_PREFIX "LabIoTMesh"
#define MESH_PASSWORD "iotmeshpassword"
#define MESH_PORT 5555
const char* nodeName = "Node-2-LDR";
const int ldrPin = A0; // Pin ADC NodeMCU
Scheduler userScheduler;
painlessMesh mesh;
void sendMessage();
Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
void sendMessage() {
int nilaiLDR = analogRead(ldrPin);
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')
New Line
115200 baud
[KIRIM MESH - NODE 2] {"tipe":"cahaya_node","adc":581}
[KIRIM MESH - NODE 2] {"tipe":"cahaya_node","adc":326}
[KIRIM MESH - NODE 2] {"tipe":"cahaya_node","adc":15}
[KIRIM MESH - NODE 2] {"tipe":"cahaya_node","adc":16}
[KIRIM MESH - NODE 2] {"tipe":"cahaya_node","adc":15}
[KIRIM MESH - NODE 2] {"tipe":"cahaya_node","adc":15}
[KIRIM MESH - NODE 2] {"tipe":"cahaya_node","adc":629}
[KIRIM MESH - NODE 2] {"tipe":"cahaya_node","adc":31}
Downloading index: package_esp8266com_index.json
Ln 1, Col 1 NodeMCU 1.0 (ESP-12E Module) on COM5

NODE 3 :


sketch_sep21a | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
LIBRARY MANAGER
Filter your search...
Type: All
Topic: All
AIPlc_Opta by Arduino
Arduino IDE PLC runtime library for Arduino Opta This is the runtime library and plugins fo...
More info
1.3.2 INSTALL
AIPlc_PMC by Arduino
Arduino IDE PLC runtime library for Arduino Portenta Machine Control This is the runtime...
More info
1.0.6 INSTALL
Arduino Cloud Provider Examples by...
Examples of how to connect
sketch_sep21a.ino
digitalWrite(ledPin, LOW); // Pastikan LED mati saat awal
mesh.setDebugMsgTypes(ERROR | STARTUP);
mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);
mesh.onReceive(&receivedCallback);
mesh.onNewConnection(&newConnectionCallback);
mesh.onChangedConnections(&changedConnectionCallback);
// Perhatikan: taskSendMessage TIDAK diaktifkan di Node 3 (berperan pasif)
Serial.println("Node 3 (Hub Aktuator Cerdas) Siap Menunggu Data...");
}
void loop() {
mesh.update();
}
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM9')
New Line
115200 baud
[TERIMA LDR] Dari 755597477 -> Nilai ADC Cahaya: 15
[RULE ENGINE] KONDISI KRITIS! LED MENYALA (HIGH)
[TERIMA LDR] Dari 2882347456 -> Nilai ADC Cahaya: 760
[RULE ENGINE] Kondisi Aman. LED MATI (LOW)
--> Topologi rantai mesh telah diperbarui (Self-Healing/Topology Change)
Ln 13, Col 45 NodeMCU 1.0 (ESP-12E Module) on COM9
100%
09.54
12/10/2026


mesh.setDebugMsgTypes(ERROR | STARTUP);
mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);
mesh.onReceive(&receivedCallback);
mesh.onNewConnection(&newConnectionCallback);
mesh.onChangedConnections(&changedConnectionCallback);
// Perhatikan: taskSendMessage TIDAK diaktifkan di Node 3 (berperan pasif)
Serial.println("Node 3 (Hub Aktuator Cerdas) Siap Menunggu Data...");
}
void loop() {
mesh.update();
}
Output Serial Monitor
[TERIMA LDR] Dari 755597477 -> Nilai ADC Cahaya: 15
[RULE ENGINE] KONDISI KRITIS! LED MENYALA (HIGH)
[TERIMA LDR] Dari 2882347456 -> Nilai ADC Cahaya: 760
[RULE ENGINE] Kondisi Aman. LED MATI (LOW)
--> Topologi rantai mesh telah diperbarui (Self-Healing/Topology Change)
AIPlc_Opta by Arduino
AIPlc_PMC by Arduino
Arduino Cloud Provider Examples